

Cache-Friendly Graph Processing Report

Pointer BFS vs CSR BFS

Setup

I ran the provided benchmark and cache simulation on the generated star graph with $n = 20000$, source vertex 0, and these repeat counts:

- Runtime rows: `repeat=100`
- Cachegrind rows: `repeat=50`

Baseline cache configuration: $I1 = 32\text{KB}$, 8-way, 64B; $D1 = 32\text{KB}$, 8-way, 64B; $LL = 8\text{MB}$, 16-way, 64B.

Q1: Runtime Comparison

From `metrics.csv` (runtime scope):

- Pointer BFS average time per run: 0.0394 ms
- CSR BFS average time per run: 0.0292 ms

CSR is faster in this experiment. The relative speedup is

$$\frac{0.0394 - 0.0292}{0.0394} \times 100 \approx 25.9\%.$$

So CSR is about $1.35\times$ faster on runtime for this test.

Q2: Cache Behavior (Baseline Cachegrind)

From baseline Cachegrind rows:

Implementation	D1 misses/run	LLd misses/run
Pointer	18 096.92	335.98
CSR	8634.96	386.00

Observations:

- CSR has far fewer $D1$ misses (about 52.3% lower than pointer).
- Pointer has slightly fewer LLd misses (about 12.96% lower than CSR).

Interpretation: CSR improves L1 data-cache locality strongly because neighbor arrays are contiguous. However, this specific graph/workload still produces similar (or slightly worse) last-level behavior for CSR.

Q3: Why CSR Tends to Perform Better

CSR usually performs better because its memory layout is contiguous:

- **Spatial locality:** Neighbor indices for one vertex are stored in adjacent memory.
- **Fewer pointer dereferences:** Traversal uses array indexing rather than linked-list pointer chasing.
- **Better hardware prefetch:** Sequential access patterns are easier for CPU prefetchers.

In contrast, pointer-based adjacency lists can scatter nodes in heap memory, increasing random access and cache pressure. Even when total algorithmic complexity is the same, memory layout changes effective performance.

Q4: Effect of Cache Parameters

1) Increasing cache size

Comparing D1 sizes 16KB $\rightarrow$ 32KB $\rightarrow$ 64KB:

- Pointer D1 misses/run: 18140.28 $\rightarrow$ 18096.92 $\rightarrow$ 18085.40 (small improvement)
- CSR D1 misses/run: 8678.32 $\rightarrow$ 8634.96 $\rightarrow$ 8623.44 (small improvement)

Larger D1 helps both implementations a little; benefit is modest for this workload.

2) Increasing associativity

With fixed 32KB, 64B D1 and associativity 1, 2, 4, 8:

- Pointer D1 misses/run: 21348.82, 18152.22, 18101.84, 18096.92
- CSR D1 misses/run: 11892.18, 8690.28, 8639.88, 8634.96

Higher associativity clearly reduces conflict misses, especially moving from direct-mapped (1-way) to 2-way.

3) Changing line size

With fixed 32KB, 8-way D1 and line sizes 32, 64, 128 B:

- Pointer D1 misses/run: 36137.78, 18096.92, 9069.14
- CSR D1 misses/run: 17224.14, 8634.96, 4333.00

Larger line size greatly lowers D1 misses for both because BFS neighbor access has spatial locality.

4) Does CSR benefit more from larger lines?

Both improve by almost the same fraction from 32B to 128B lines:

- Pointer: about 74.9% reduction in D1 misses/run
- CSR: about 74.8% reduction in D1 misses/run

So in this dataset, both benefit similarly in percentage terms. CSR still has fewer absolute misses at each line size.

Conclusion

The collected metrics match the assignment requirements and support the expected memory-layout lesson: contiguous storage (CSR) substantially reduces L1 data-cache misses and improves runtime in this experiment. Cache size has small impact, while associativity and especially line size have stronger effects.